

1. User 2 will still be able to read the file because only user 1's directory entry to the file was removed, user 2's entry still exists.
2. Multiprogramming is likely to increase CPU utilization when many processes are I/O bound and are blocked waiting for I/O. So instead sitting idle, the CPU can accomplish other tasks by letting other processes make progress.
3. Half capacity: $1,000,000,000 / 2 = 500,000,000$ instructions/sec, 1 sys call = 1000 instructions, $500,000,000 / 1000 = 500,000$ system calls / sec
4. A race condition is when two or more processes access shared data, and at least one of them modifies it, and the final result depends on the timing/interleaving of their execution. Symptoms include inconsistent/wrong results changing from run to run, unpredictable behavior, etc.
5. The solution is safe because it meets the first two requirements which are that only one process is allowed in the critical section at one time, and that there can be no assumptions made about the number of CPUs and the speeds of them. So adding another CPU wouldn't change how the program runs.
6. During run time, user calls down() or up() on the semaphore, then control switches to the OS which disables interrupts. Then the down() or up() logic executes, either updating the semaphore or putting thread to sleep/wake sleeping thread. Then when the semaphore internal state changes are fully consistent the OS enables interrupts again. If the calling thread was not blocked, execution continues. If it was, the OS runs another runnable thread until the blocked one can run.
7. If a process occurred twice in the Round Robin list, it would just be picked to run/progress twice as many times as the other processes in the list. One reason for allowing this is to give that process a higher priority, since Round Robin implicitly determines all processes to have the same priority.
8. Batch job name: (running time, priority (5 is highest), quantum):
 - A: (10, 3, 1)
 - B: (3, 5, 1)
 - C: (4, 2, 1)
 - D: (7, 1, 1)
 - E: (6, 4, 1)

a) Round robin:

ABCDE ABCDE ABCDE ACDE ADE ADE AD A A A

- Finishing times:

- Process A: $5 + 5 + 5 + 4 + 3 + 3 + 2 + 1 + 1 + 1 = 30$ seconds
- Process B: $5 + 5 + 2 = 12$ seconds
- Process C: $5 + 5 + 5 + 2 = 17$ seconds
- Process D: $5 + 5 + 5 + 4 + 3 + 3 + 2 = 27$ seconds
- Process E: $5 + 5 + 5 + 4 + 3 + 3 = 25$ seconds
- Mean process turnaround time: $(30 + 12 + 17 + 27 + 25) / 5 = 22.2$ seconds

b) Priority scheduling:

BBB EEEEE AAAAAAAAAA CCCC DDDDDDD

- Finishing times:

- Process A: $3 + 6 + 10 = 19$ seconds

- Process B: 3 seconds
- Process C: $3 + 6 + 10 + 4 = 23$ seconds
- Process D: $3 + 6 + 10 + 4 + 7 = 30$ seconds
- Process E: $3 + 6 = 9$ seconds
- Mean process turnaround time: $(19 + 3 + 23 + 30 + 9) / 5 = 16.8$

c) First-come, first served (arrived in alphabetical order)

AAAAAAAAAA BBB CCCC DDDDDDD EEEEEEE

- Finishing times:
 - Process A: 10 seconds
 - Process B: $10 + 3 = 13$ seconds
 - Process C: $10 + 3 + 4 = 17$ seconds
 - Process D: $10 + 3 + 4 + 7 = 24$ seconds
 - Process E: $10 + 3 + 4 + 7 + 6 = 30$ seconds
- Mean process turnaround time: $(10 + 13 + 17 + 24 + 30) / 5 = 18.8$ seconds

d) Shortest job first

BBB CCCC EEEEEEE DDDDDDD AAAAAAAAAA

- Finishing times:
 - Process A: $3 + 4 + 6 + 7 + 10 = 30$ seconds
 - Process B: 3 seconds
 - Process C: $3 + 4 = 7$ seconds
 - Process D: $3 + 4 + 6 + 7 = 20$ seconds
 - Process E: $3 + 4 + 6 = 13$ seconds
- Mean process turnaround time: $(30 + 3 + 7 + 20 + 13) / 5 = 14.6$ seconds

9. 8a modified: tie break, put D first if finish at same time as quantum

- Batch job name: (running time, priority (5 is highest), quantum):
 - A: (10, 3, 1)
 - B: (3, 5, 1)
 - C: (4, 2, 1)
 - D: (7, 1, 0.5), blocks for 1 second
 - E: (6, 4, 1)

a) Round robin:

ABCDE ABCDE ABCDE ACDE ADE ADE AD AD AD AD X D X D X D X D

- Finishing times:
 - Process A: $4.5 * 3 + 3.5 + 2.5 * 2 + 1.5 * 3 + 1 = 27.5$ seconds
 - Process B: $4.5 * 2 + 2 = 11$ seconds
 - Process C: $4.5 * 3 + 2 = 15.5$ seconds
 - Process D: $4.5 * 3 + 3.5 + 2.5 * 2 + 1.5 * 4 + 1.5 * 4 = 34$ seconds
 - Process E: $4.5 * 3 + 3.5 + 2.5 * 2 = 22$ seconds
- Mean process turnaround time: $(27.5 + 11 + 15.5 + 34 + 22) / 5 = 22$ seconds

10. It would take 5 swaps including initial load

- 1) Given 1 quantum: $30 - 1 = 29$ quanta remaining
- 2) Given 2 quanta: $29 - 2 = 27$ quanta remaining
- 3) Given 4 quanta: $27 - 4 = 23$ quanta remaining
- 4) Given 8 quanta: $23 - 8 = 15$ quanta remaining

Julia Lu

5) Given 16 quanta: $15 - 16 = -1$, 0 quanta remaining